



AROR UNIVERSITY
OF ART, ARCHITECTURE,
DESIGN & HERITAGE,
SUKKUR, SINDH

Software Evolution

Prof. Dr. Fahim Akhtar Rajput





Software change

- Software change is inevitable
 - New requirements emerge when the software is used;
 - The business environment changes;
 - Errors must be repaired;
 - New computers and equipment is added to the system;
 - The performance or reliability of the system may have to be improved.
- A key problem for all organizations is implementing and managing change to their existing software systems.



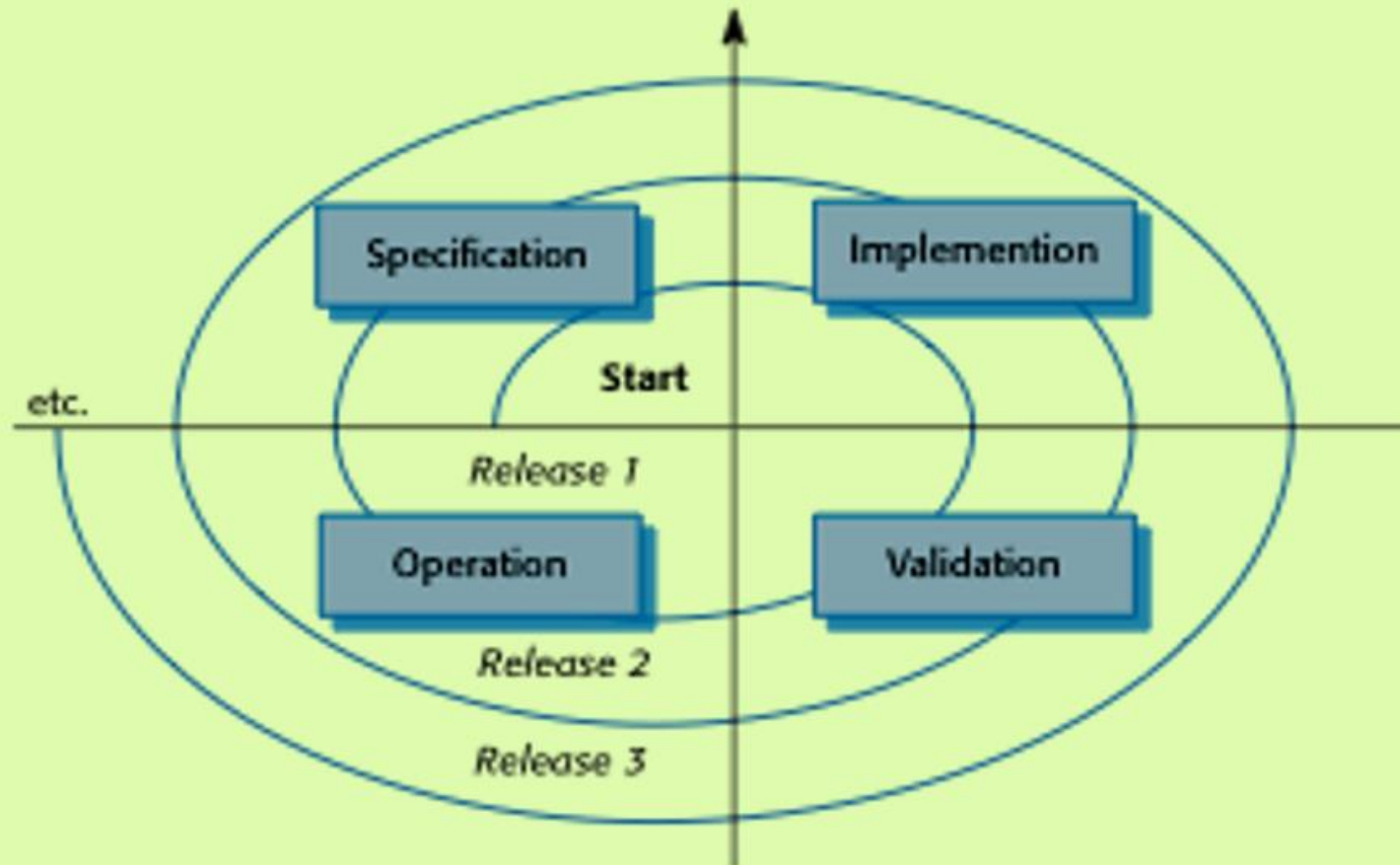
Importance of evolution

- Organizations have huge investments in their software systems - they are critical business assets.
- To maintain the value of these assets to the business, they must be changed and updated.
- The majority of the software budget in large companies is devoted to changing and evolving existing software rather than developing new software.

Importance of evolution

- The requirements of installed software systems change as the business and its environment change, so new releases of the systems that incorporate changes and updates are usually created at regular intervals.
- Software engineering is therefore a spiral process with requirements, design, implementation, and testing going on throughout the lifetime of the system (Figure 9.1).
- You start by creating release 1 of the system.
- Once delivered, changes are proposed, and the development of release 2 starts almost immediately.
- In fact, the need for evolution may become obvious even before the system is deployed, so later releases of the software may start development before the current version has even been released.

A spiral model of development and evolution



Applicability of Spiral Model

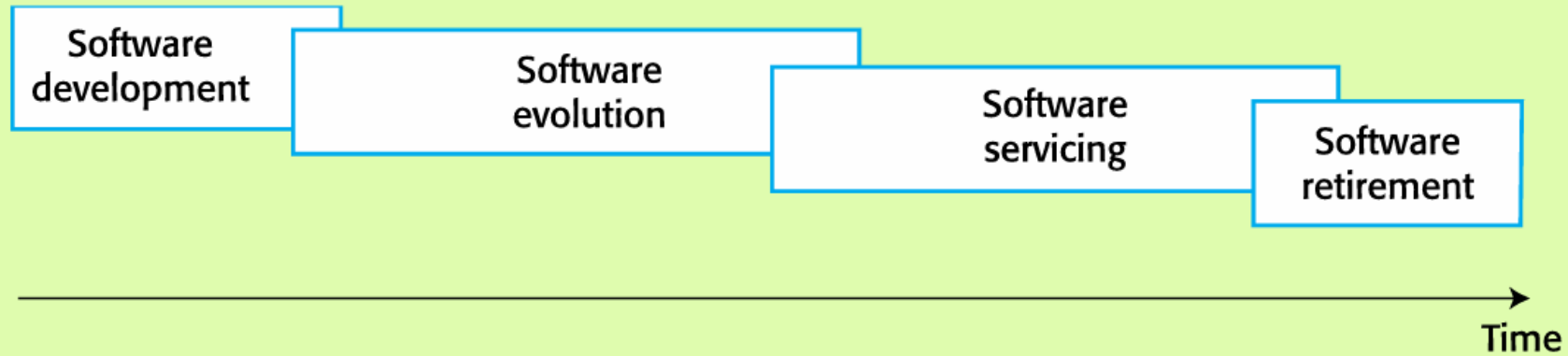
- This model of software evolution is applicable when the same company is responsible for the software throughout its lifetime.
 - There is a seamless transition from development to evolution, and the same software development methods and processes are applied throughout the lifetime of the software.
 - Software products and apps are developed using this approach.
- The evolution of custom software, however, usually follows a different model.
 - The system customer may pay a software company to develop the software and then take over responsibility for support and evolution using its own staff.
 - Alternatively, the software customer might issue a separate contract to a different software company for system support and evolution.



Evolution and servicing

- Rajlich and Bennett (Rajlich and Bennett 2000) propose an alternative view of the software evolution life cycle for business systems.
- In this model, they distinguish between servicing, evolution and servicing.
- Evolution is the phase in which significant changes to the software architecture and functionality are made.
- During servicing, the only changes that are made are relatively small but essential changes.
- These phases overlap with each other, as shown in Figure 9.2.

Evolution and servicing





Evolution and servicing

- **Evolution**

- The stage in a software system's life cycle where it is in operational use and is evolving as new requirements are proposed and implemented in the system.

- **Servicing**

- At this stage, the software remains useful but the only changes made are those required to keep it operational i.e. bug fixes and changes to reflect changes in the software's environment. No new functionality is added.

- **Phase-out**

- The software may still be used but no further changes are made to it.



Evolution processes

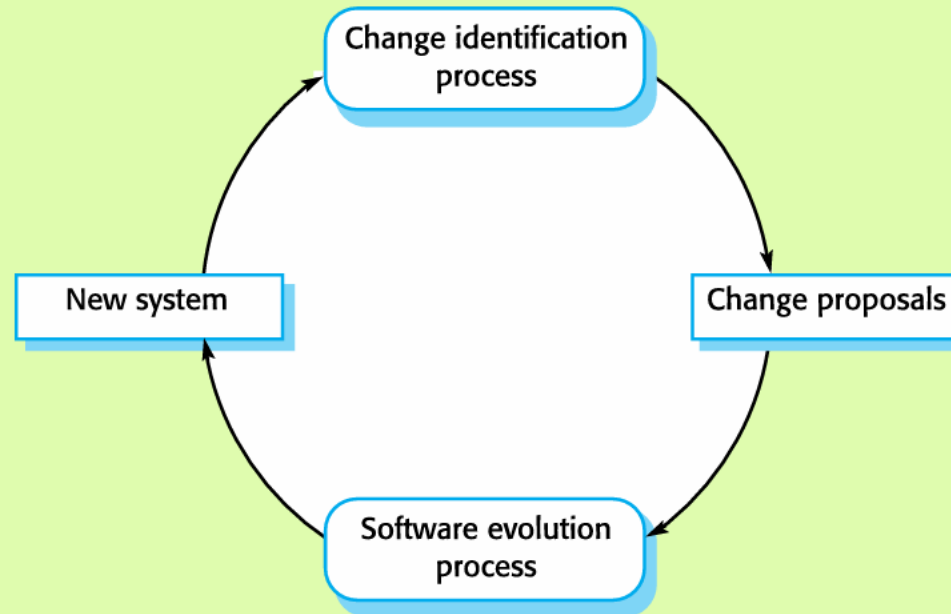
- The most appropriate evolution process for a software system depends on the type of software being maintained, the software development processes used in an organization, and the skills of the people involved.
- For some types of system, such as mobile apps, evolution may be an informal process, where change requests mostly come from conversations between system users and developers.
- For other types of systems, such as embedded critical systems, software evolution may be formalized, with structured documentation produced at each stage in the process.



Evolution processes

- Formal or informal system change proposals are the driver for system evolution in all organizations.
- In a change proposal, an individual or group suggests changes and updates to an existing software system.
- These proposals may be based on existing requirements that have not been implemented in the released system, requests for new requirements, bug reports from system stakeholders, and new ideas for software improvement from the system development team.
- The processes of change identification and system evolution are cyclical and continue throughout the lifetime of a system (Figure 9.3).

Change identification and evolution processes





Evolution processes

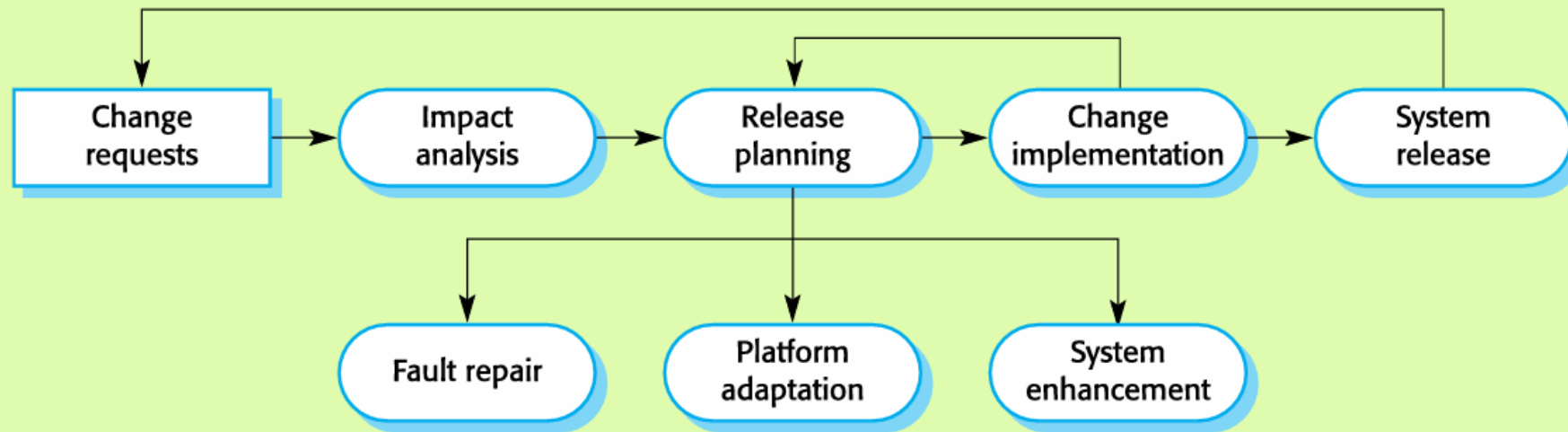
- Software evolution processes depend on
 - The type of software being maintained;
 - The development processes used;
 - The skills and experience of the people involved.
- Proposals for change are the driver for system evolution.
 - Should be linked with components that are affected by the change, thus allowing the cost and impact of the change to be estimated.
- Change identification and evolution continues throughout the system lifetime.



Evolution processes

- Before a change proposal is accepted, there needs to be an analysis of the software to work out which components need to be changed.
- This analysis allows the cost and the impact of the change to be assessed.
- This is part of the general process of change management, which should also ensure that the correct versions of components are included in each system release.

The software evolution process



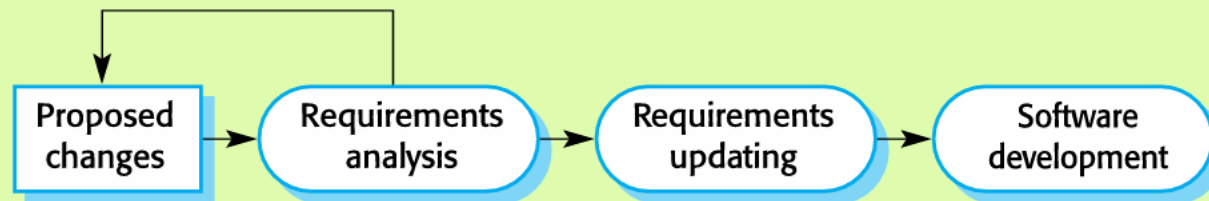
- The process includes the fundamental activities of change analysis, release planning, system implementation, and releasing a system to customers.
- The cost and impact of these changes are assessed to see how much of the system is affected by the change and how much it might cost to implement the change.



Change implementation

- Where different teams are involved, a critical difference between development and evolution is that the first stage of change implementation requires program understanding.
- During the program understanding phase, new developers have to understand how the program is structured, how it delivers functionality, and how the proposed change might affect the program.
- They need this understanding to make sure that the implemented change does not cause new problems when it is introduced into the existing system.
- If requirements specification and design documents are available, these should be updated during the evolution process to reflect the changes that are required (Figure 9.5).

Change implementation



Urgent change requests

- Urgent changes may have to be implemented without going through all stages of the software engineering process.
 - If a serious system fault has to be repaired to allow normal operation to continue or to address a serious security vulnerability.
 - If changes to the system's environment (e.g. an OS upgrade) have unexpected effects that disrupt normal operation.
 - If there are unanticipated changes to the business running the system, such as the emergency of new competitors or the introduction of new legislation that affect the system.

The emergency repair process

In these cases, the need to make the change quickly means that you may not be able to update all of the software documentation. Rather than modify the requirement and design, you make an emergency fix to the program to solve the immediate problem.



Emergency system repairs have to be completed as quickly as possible. You choose a quick and workable solution rather than the best solution as far as system structure is concerned. This tends to accelerate the process of software ageing so that future changes become progressively more difficult and maintenance costs increase.



Legacy systems

- Legacy systems are older systems that rely on languages and technology that are no longer used for new systems development.
- Legacy software may be dependent on older hardware, such as mainframe computers and may have associated legacy processes and procedures.
- Legacy systems are not just software systems but are broader socio-technical systems that include hardware, software, libraries and other supporting software and business processes.

The elements of a legacy system

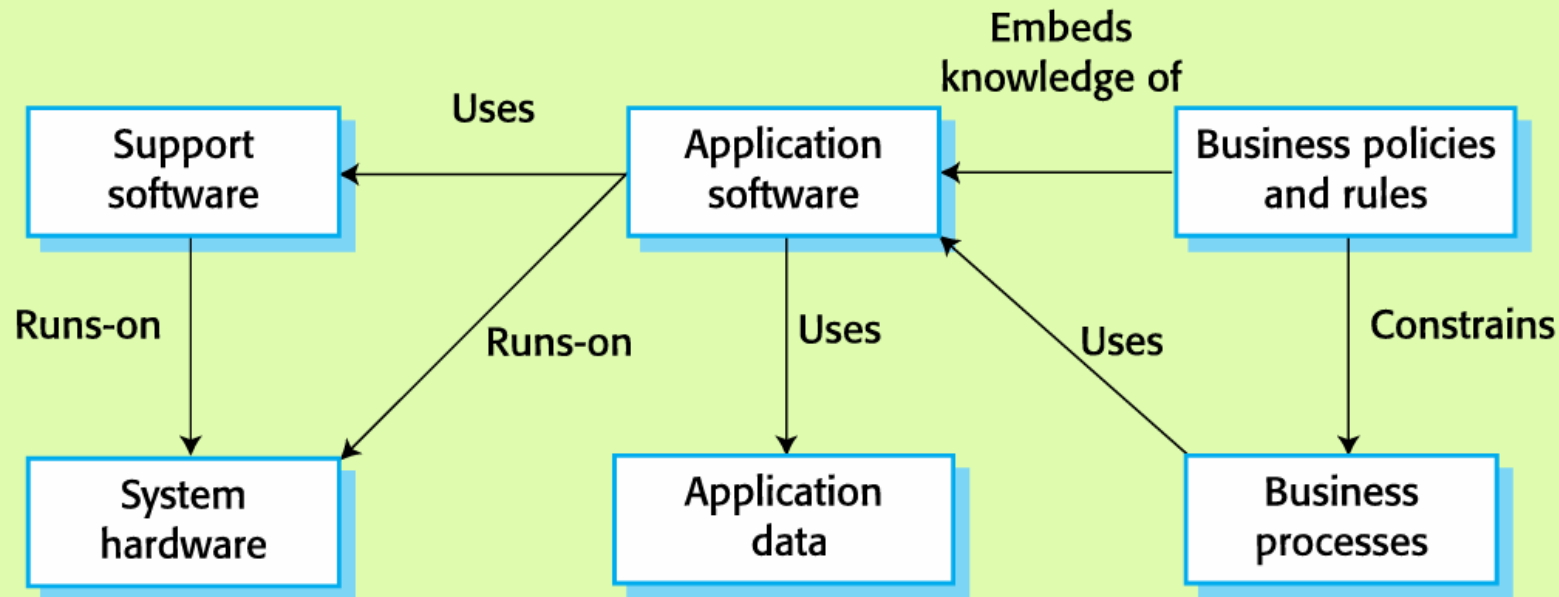


Figure 9.7 shows the logical parts of a legacy system and their relationships.



Legacy system components

- *System hardware* Legacy systems may have been written for hardware that is no longer available.
- *Support software* The legacy system may rely on a range of support software, which may be obsolete or unsupported.
- *Application software* The application system that provides the business services is usually made up of a number of application programs.
- *Application data* These are data that are processed by the application system. They may be inconsistent, duplicated or held in different databases.

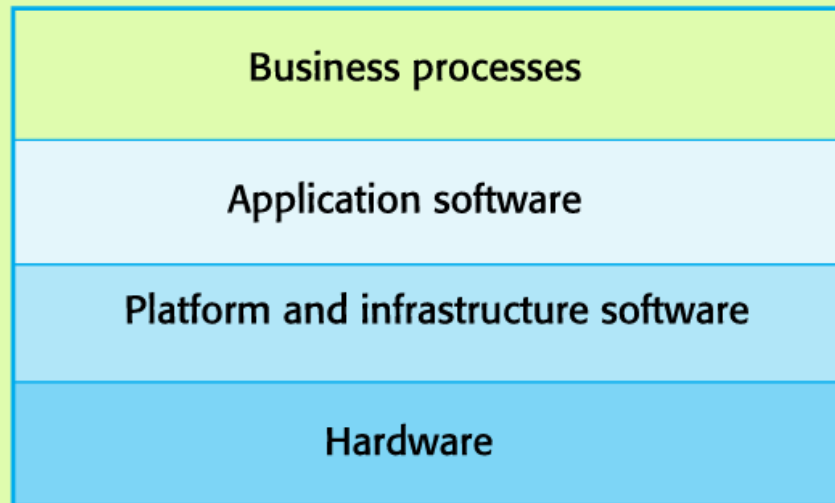
Legacy system components (Cont.)

- *Business processes* These are processes that are used in the business to achieve some business objective.
 - Business processes may be designed around a legacy system and constrained by the functionality that it provides.
- *Business policies and rules* These are definitions of how the business should be carried out and constraints on the business. Use of the legacy application system may be embedded in these policies and rules.

Legacy system layers

An alternative way of looking at these components of a legacy system is as a series of layers, as shown in Figure 9.8.

Socio-technical system



Each layer depends on the layer immediately below it and interfaces with that layer.

If interfaces are maintained, then you should be able to make changes within a layer without affecting either of the adjacent layers.

In practice, however, this simple encapsulation is an oversimplification, and changes to one layer of the system may require consequent changes to layers that are both above and below the changed level.

Legacy system Layers (Cont.)

- Changing one layer in the system may introduce new facilities, and higher layer in the system may then be changed to take advantage of these facilities.
- Changing the software may slow the system down so that new hardware is needed to improve the system performance.
 - The increase in performance from the new hardware may then mean that further software changes that were previously impractical become possible.
- It is often impossible to maintain hardware interfaces, especially if new hardware is introduced.
 - This is particular problem in embedded system where there is a tight coupling between software and hardware. As a result, major changes to the application software may be required to make effective use of the new hardware.



Legacy system replacement

- Legacy system replacement is risky and expensive so businesses continue to use these systems
- System replacement is risky for a number of reasons
 - Lack of complete system specification
 - Tight integration of system and business processes
 - Undocumented business rules embedded in the legacy system
 - New software development may be late and/or over budget



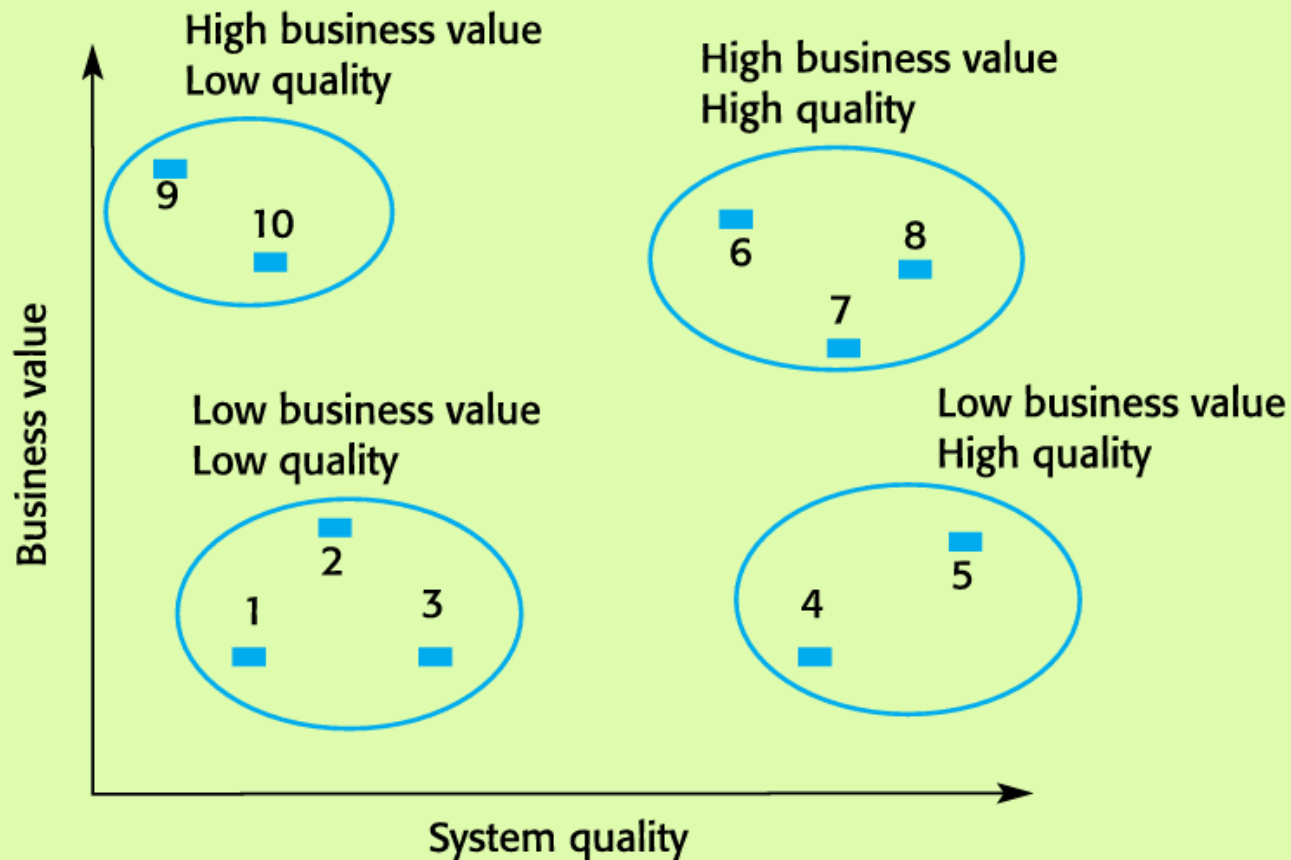
Legacy system change

- Legacy systems are expensive to change for a number of reasons:
 - No consistent programming style
 - Use of obsolete programming languages with few people available with these language skills
 - Inadequate system documentation
 - System structure degradation
 - Program optimizations may make them hard to understand
 - Data errors, duplication and inconsistency

Legacy system management

- Organisations that rely on legacy systems must choose a strategy for evolving these systems
 - Scrap the system completely and modify business processes so that it is no longer required
 - This option should be chosen when the system is not making an effective contribution to business processes.
 - Leave the system unchanged and continue with regular maintenance of the system
 - This option should be chosen when the system is still required but it is fairly stable and the system users make relatively few change request.
 - Re-engineering the system to improve its maintainability
 - This option should be chosen when the system quality has been degraded by change and where new change to the system is still being proposed.
 - Replace all or part of the system with a new system
 - This option should be chosen when factors, such as new hardware, mean that the old system cannot continue in operation, or where off-the-shelf system would allow the new system to be developed at a reasonable cost.
- The strategy chosen should depend on the system quality and its business value.

An example of a legacy system assessment



- Assume that an organization has 10 legacy systems.
- One should assess the quality and the business value of each of these systems.
- One may then create a chart showing relative business value and system quality.

Legacy system categories

- Low quality, low business value
 - Keeping these systems in operation will be expensive, and the rate of the return to the business will be fairly small.
 - These systems should be scrapped.
- Low quality, high business value
 - These systems are making an important business contribution, so they cannot be scrapped.
 - However, their low quality means that they are expensive to maintain.
 - These systems should be reengineered to improve their quality.
 - They may be replaced, if suitable off-the-shelf systems are available

Legacy system categories

- High quality, low business value
 - These systems don't contribute much to the business but may not be very expensive to maintain.
 - It is not worth replacing these systems, so normal system maintenance may be continued if expensive changes are not required and the system hardware remains in use.
 - If expensive changes become necessary, the software should be scrapped.
- High quality, high business value
 - These systems have to be kept in operation.
 - However, their high quality means that you don't have to invest in transformation or system replacement.
 - Normal system maintenance should be continued

Legacy system categories

- Low quality, low business value
 - These systems should be scrapped.
- Low quality, high business value
 - These make an important business contribution but are expensive to maintain. Should be re-engineered or replaced if a suitable system is available.
- High quality, low business value
 - Replace with COTS, scrap completely or maintain.
- High quality, high business value
 - Continue in operation using normal system maintenance.



Key points

- Software development and evolution can be thought of as an integrated, iterative process that can be represented using a spiral model.
- For custom systems, the costs of software maintenance usually exceed the software development costs.
- The process of software evolution is driven by requests for changes and includes change impact analysis, release planning and change implementation.



Key points

- Legacy systems are older software systems, developed using obsolete software and hardware technologies, that remain useful for a business.
- It is often cheaper and less risky to maintain a legacy system than to develop a replacement system using modern technology.
- The business value of a legacy system and the quality of the application should be assessed to help decide if a system should be replaced, transformed or maintained.

A decorative pattern of hexagons in various shades of blue, orange, and white, arranged in a honeycomb-like structure on the left side of the slide.

Thank you

Prof. Dr. Fahim Akhtar Rajput